Engineer to Excel

Choosing the Right Data Structure

Presented by:
Amrissh P - 192572026

0308 Data Structures

Date: 09-10-2026

Engineer to Excel

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Technical Presentation

COURSE NAME: Data Structures **COURSE CODE: CSA0308**

Register Number	**192572026**
Name	**Amrissh P**

Assessment Tool Weightage: 30%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions:4 **EACH QUESTIONS CARRIES:** 25 MARKS **Total Marks:**100 **Duration:** ONE HOUR

S. No.	List of Questions
1	A student management system needs to store the marks of 1000 students efficiently. Apply the concepts of primitive and non-primitive data structures to select an appropriate structure for storing and managing the data and prepare a technical presentation justifying your choice based on the efficiency of data structure.
2	Choose a suitable linear or non-linear data structure for a navigation system that stores city connections and finds shortest routes between locations. Prepare a technical presentation explaining why it is appropriate.
3	A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
4	A digital library allows searching books using ISBN numbers quickly among 50,000 records. Select the most suitable data structure and Prepare a technical presentation justifying using time complexity comparison.

Code of Conduct:

I 192572026 / Amrissh P (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

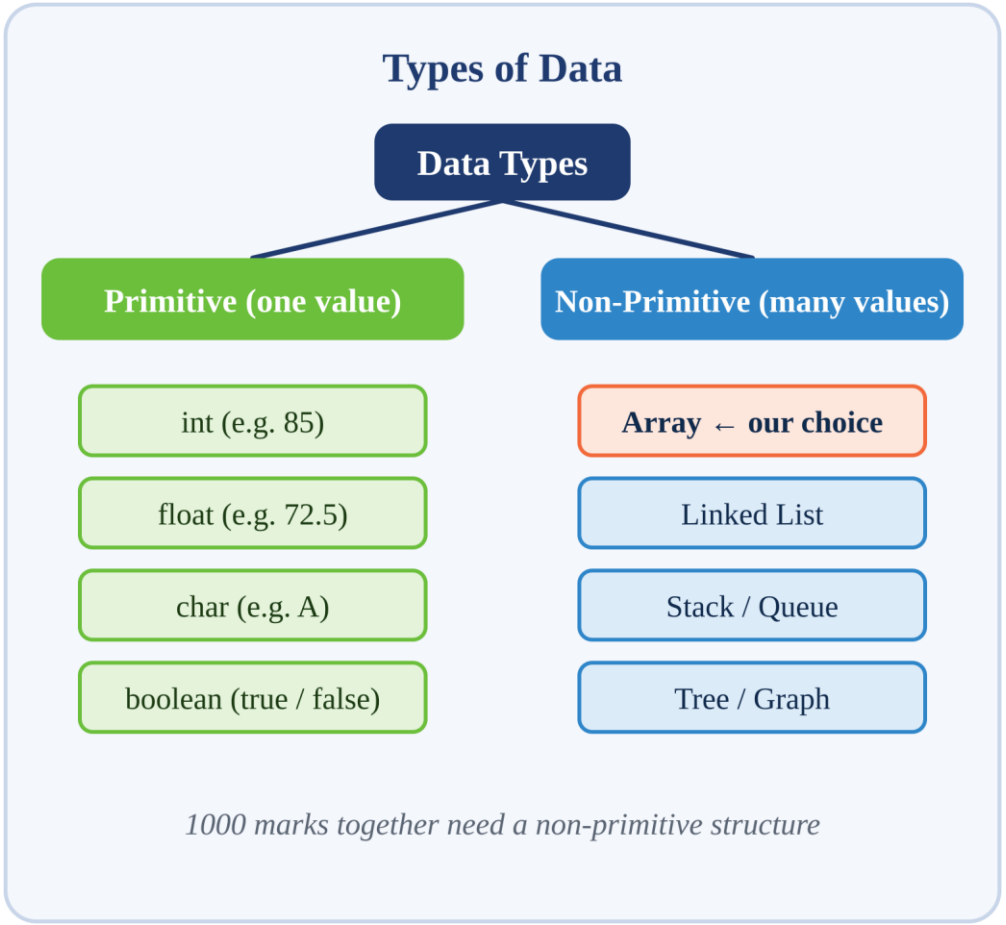


RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

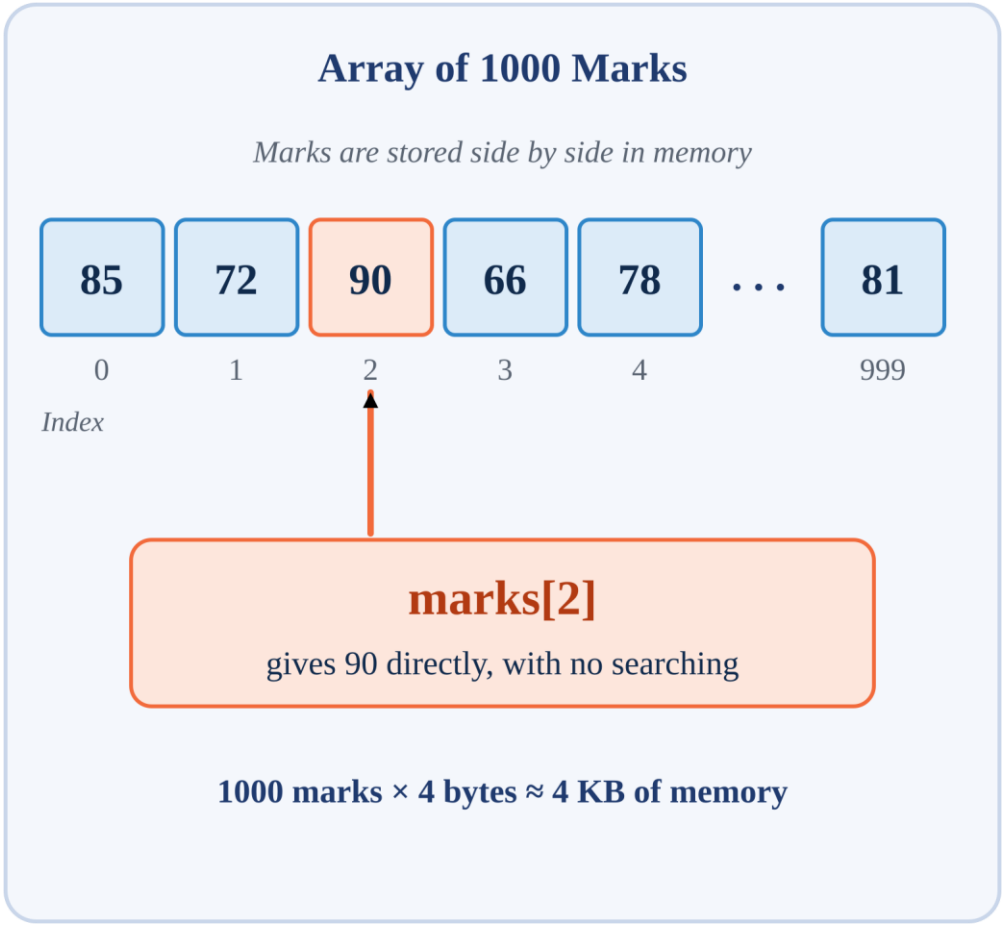
Q1: Marks of 1000 Students - Problem

- A student system must store the **marks of 1000 students**.
- **Primitive** types hold one simple value, like int or float.
- **Non-primitive** structures hold many values together, like an array.
- One mark is primitive, but 1000 marks together need a non-primitive structure.



Q1: Choosing the Data Structure

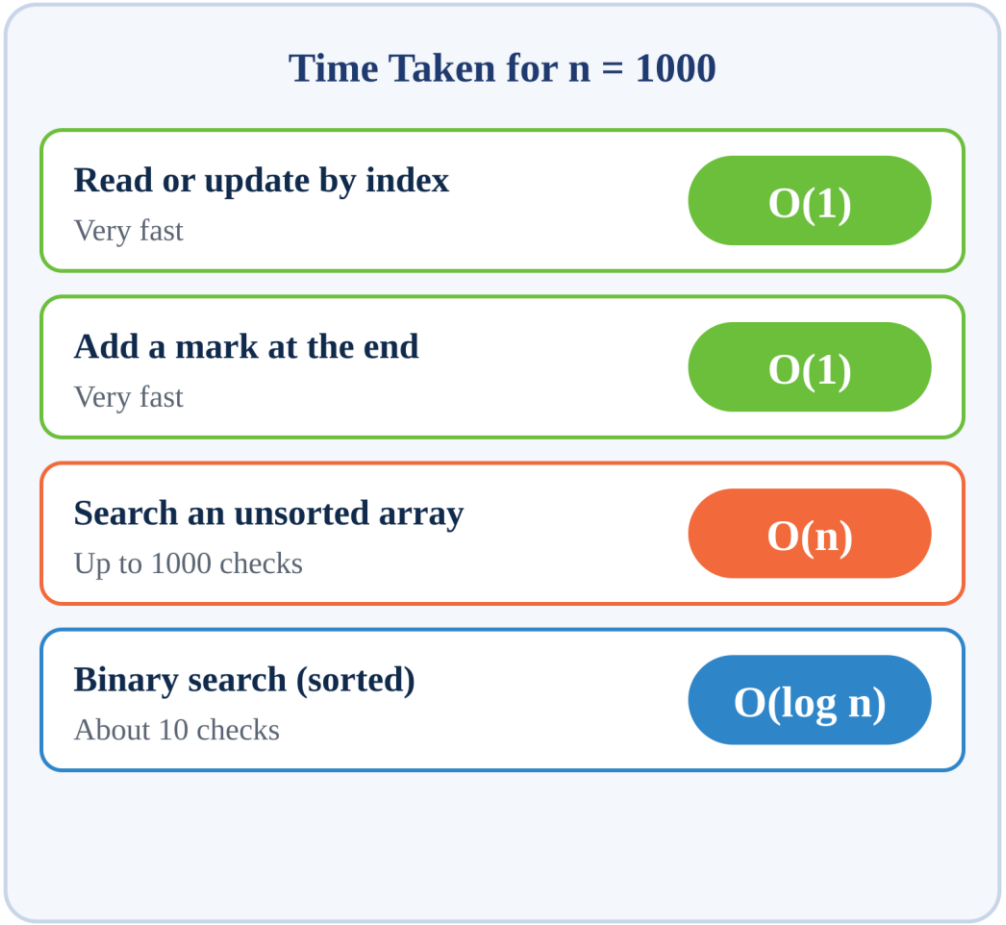
- Best choice: a **one-dimensional array** of 1000 numbers.
- The size is fixed and known, so an array fits well.
- Marks sit side by side in memory.
- Any mark is reached by its **index**, for example marks[2].
- Easy to read, update and add up all marks.



Engineer to Excel

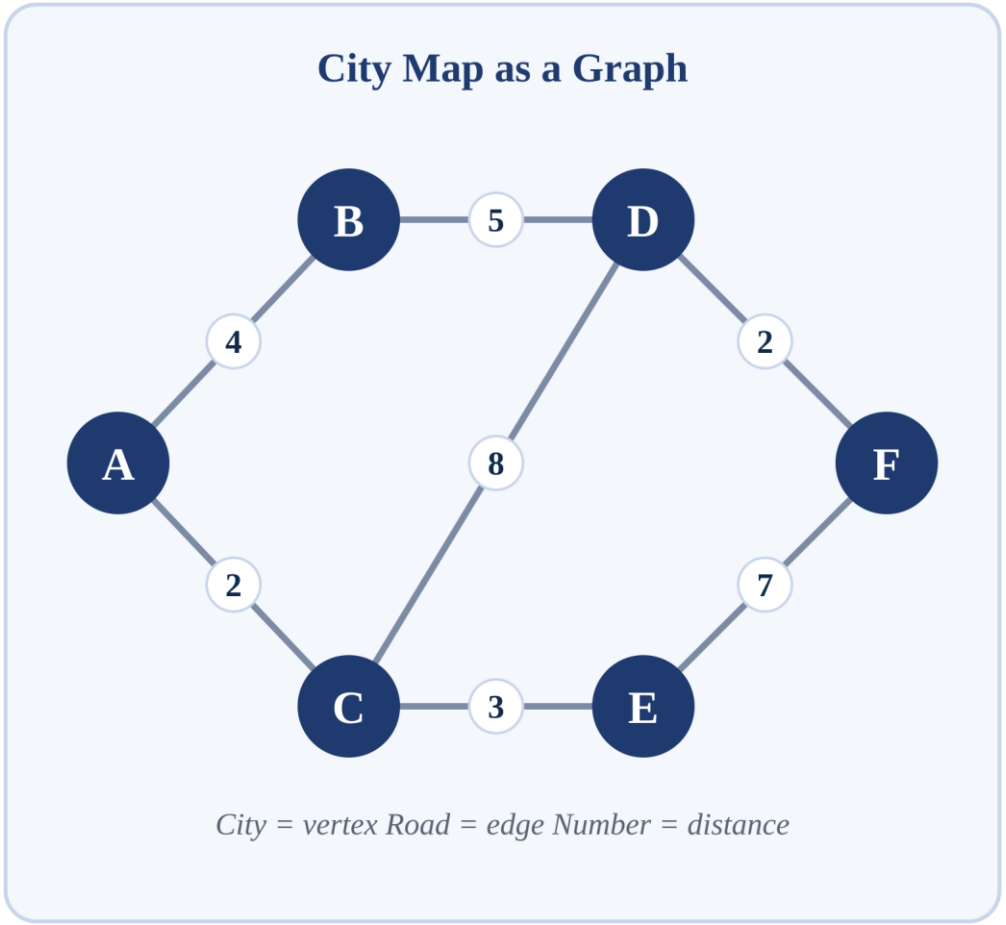
Q1: Efficiency of the Array

- Read or update a mark: **O(1)**, very fast.
- Add a mark at the end: **O(1)**.
- Search an unsorted array: **O(n)**, up to 1000 checks.
- Sorted array with binary search: **O(log n)**, about 10 checks.
- Uses little memory: about 4 KB, with no extra pointers.



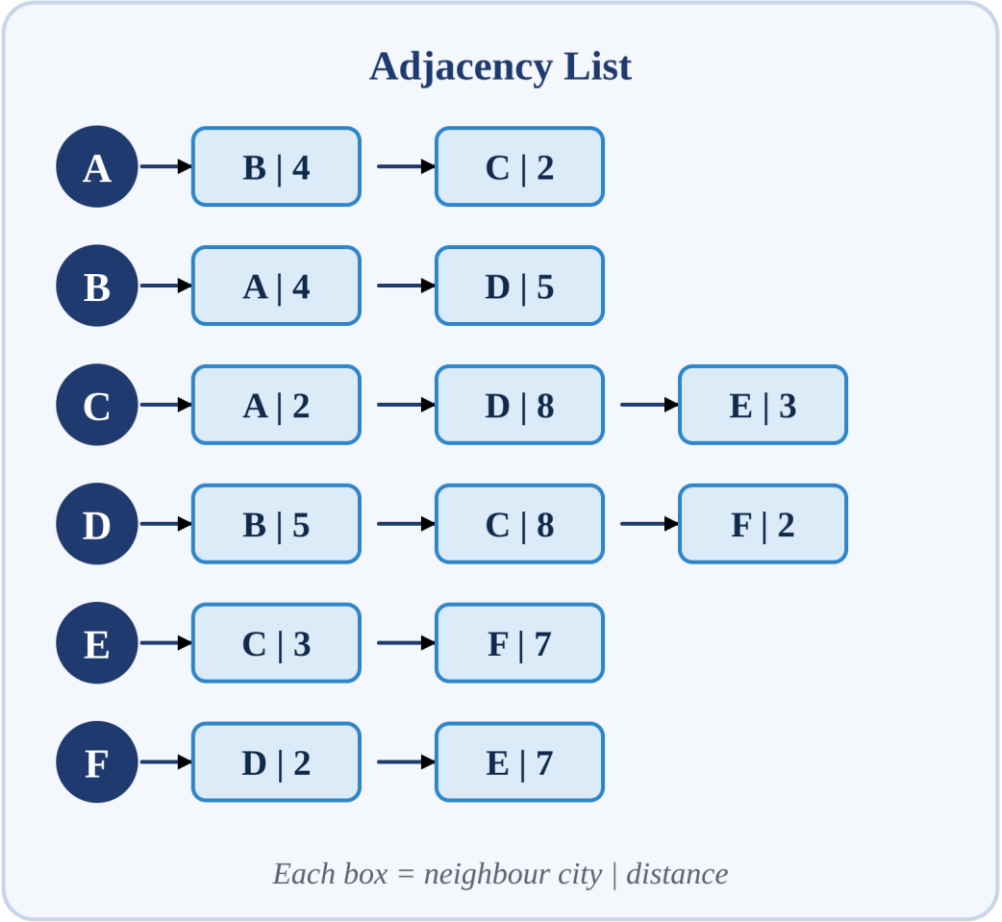
Q2: City Navigation - Problem

- The system stores **city connections** and finds the **shortest route**.
- Cities are joined by roads of different lengths.
- This is a network, not a simple list.
- Best choice: a **Graph**, a non-linear data structure.
- City = vertex, road = edge, distance = weight.



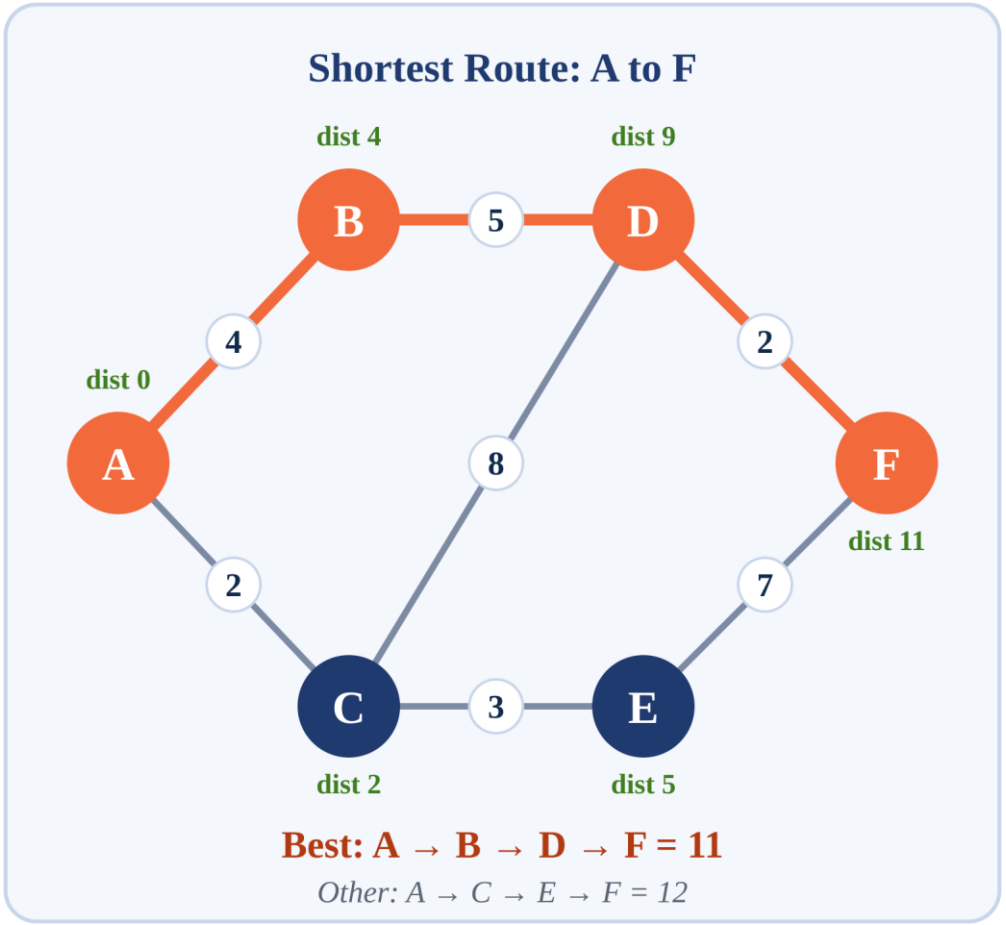
Q2: Storing the Connections

- Use an **adjacency list**.
- Each city keeps a list of its neighbour cities.
- Each entry also stores the road distance.
- Space needed is **O(V + E)**, so only real roads are stored.
- Better than a matrix for maps, as a city links to only a few others.



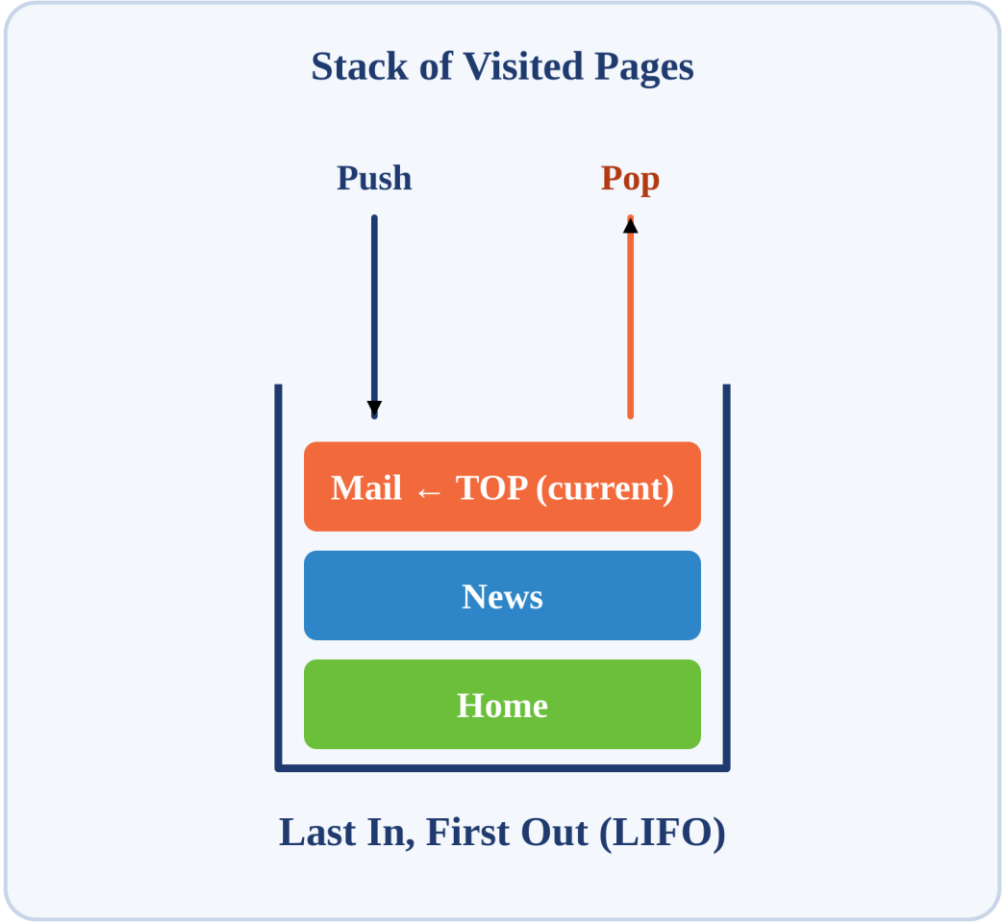
Q2: Finding the Shortest Route

- Use **Dijkstra's algorithm** on the weighted graph.
- Start city = 0 distance, all others = infinity.
- Pick the nearest unvisited city and update its neighbours.
- Stop when the destination is reached.
- Time: **O((V + E) log V)** with a priority queue.



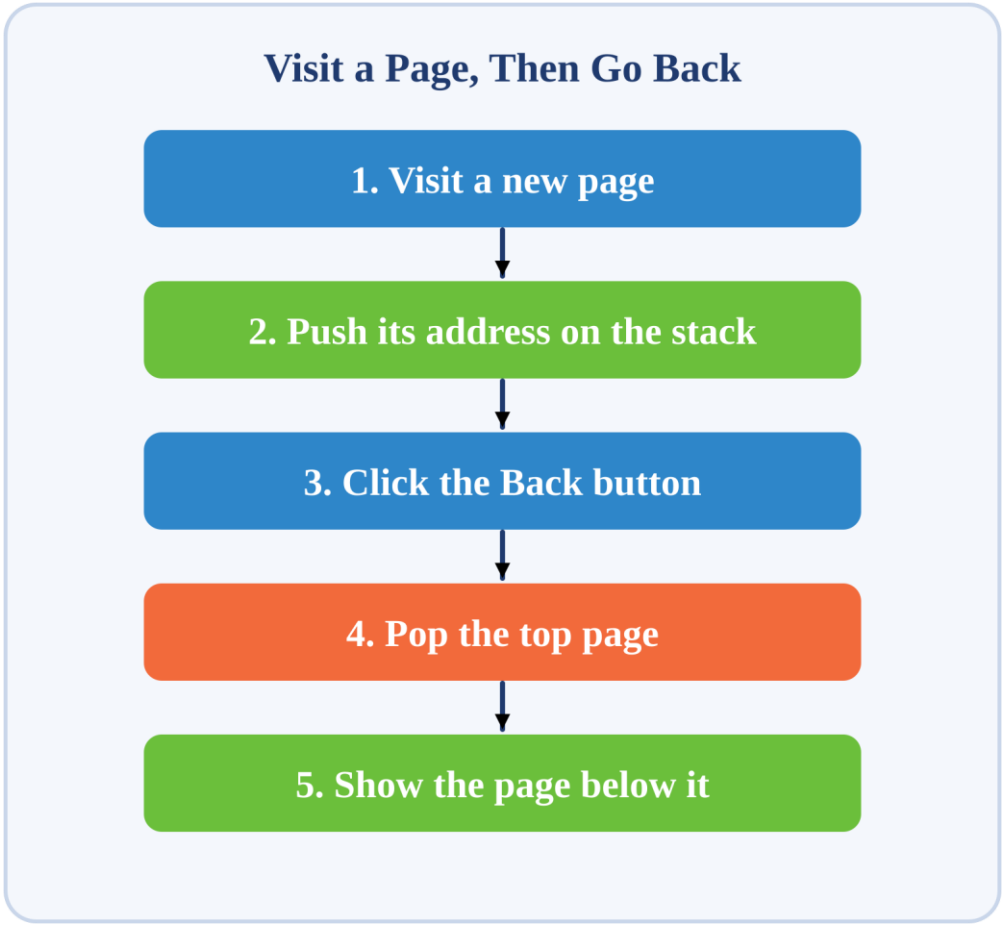
Q3: Browser Back Button - Problem

- A browser must go back to pages visited earlier.
- The **last page visited** must be shown **first**.
- This is **Last In, First Out (LIFO)**.
- Best ADT: a **Stack**.
- An ADT tells what the operations do, not how they are built.



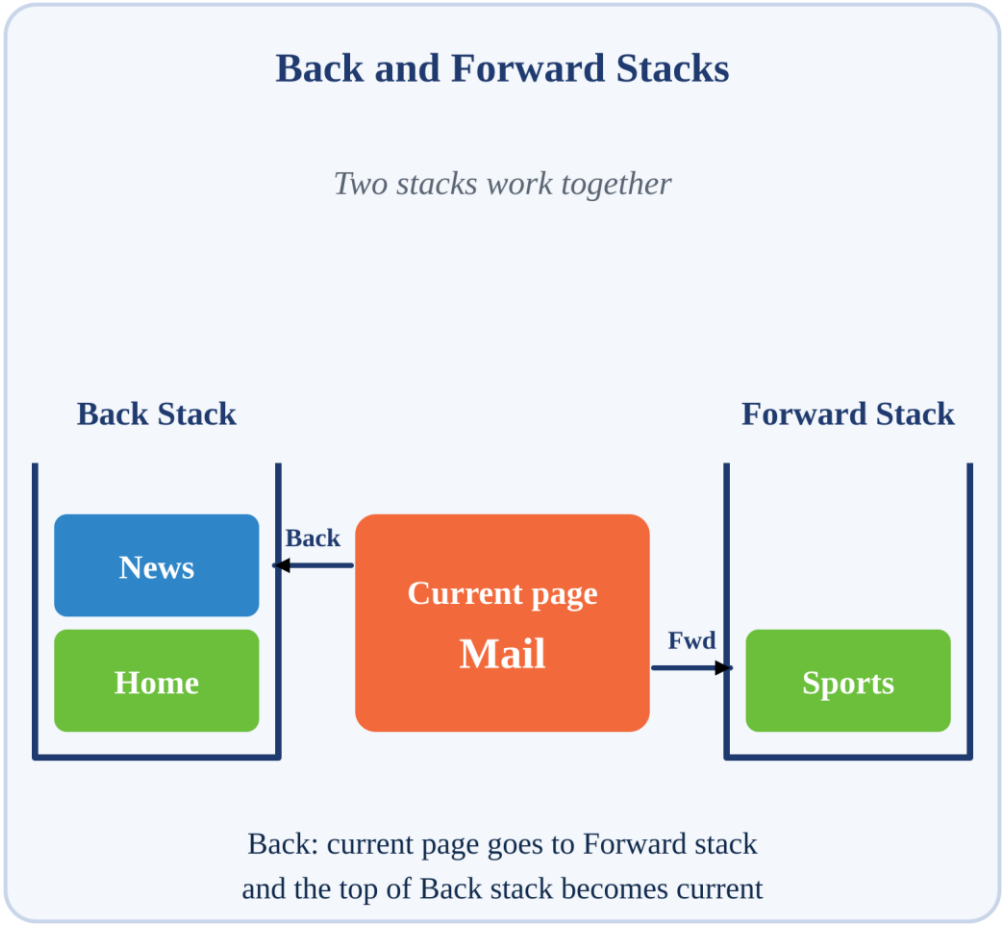
Q3: How the Stack Works

- **Push**: add a new page address on top when you visit it.
- **Pop**: remove the top page when you click Back.
- **Peek**: see the current page without removing it.
- Only the top is used, so no searching is needed.
- Example: Home, News, Mail. Back shows News, then Home.



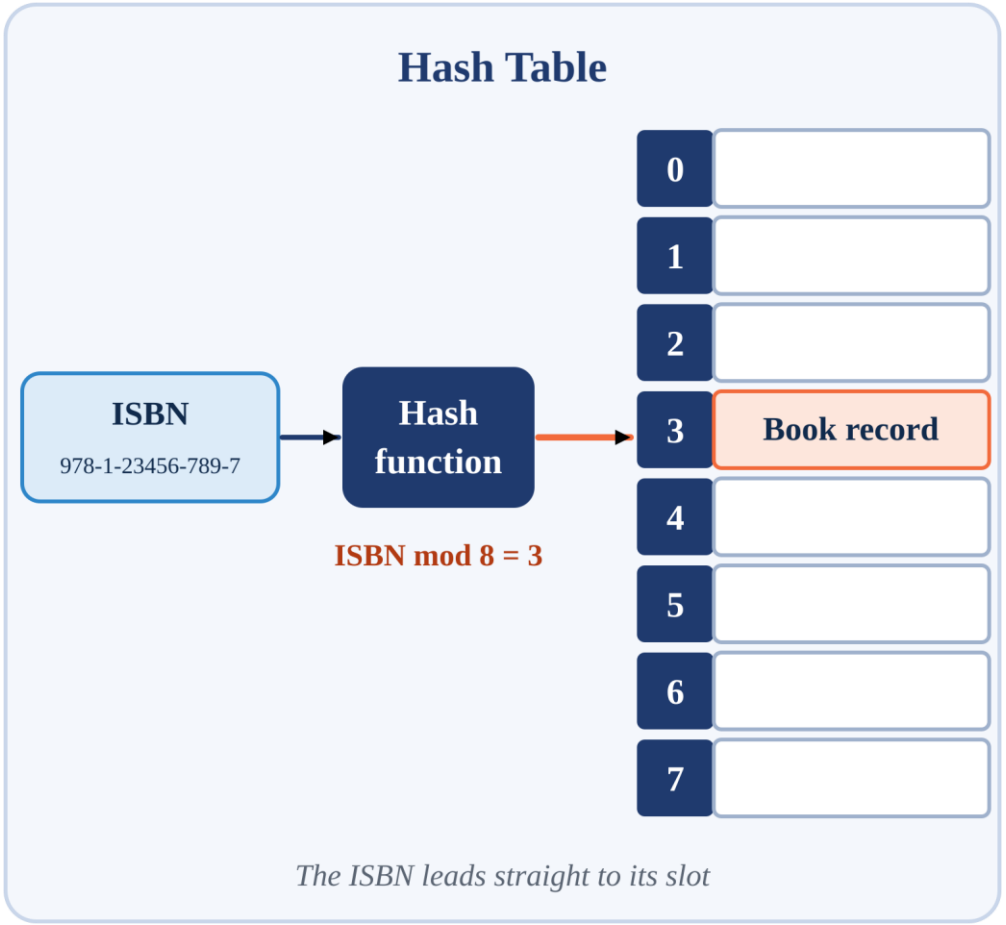
Q3: Why the Stack Is Efficient

- Push, pop and peek all take **O(1)** time.
- No shifting or searching of old pages.
- Memory is used only for pages visited.
- A second stack can handle the **Forward** button.
- Result: fast and simple navigation, even in long sessions.



Q4: ISBN Search - Problem and Choice

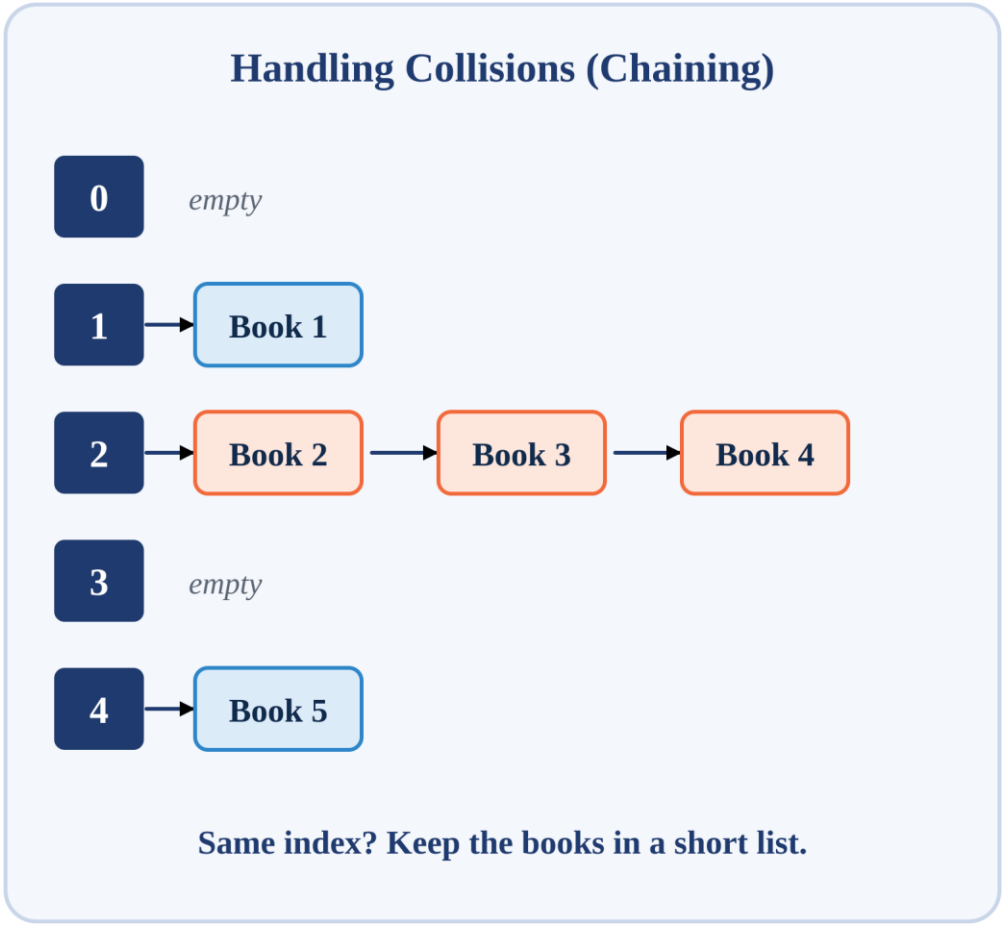
- A digital library has **50,000 books**, each with a unique ISBN.
- Searching must be very fast.
- Best choice: a **Hash Table**.
- ISBN is the **key**; the book record is the **value**.
- A hash function turns the ISBN into a table position.



Engineer to Excel

Q4: How Hashing Works

- Hash function example: **ISBN mod table size**.
- **Insert**: find the index and store the record there.
- **Search**: find the same index and go straight to it.
- **Collision**: two ISBNs get the same index. Chaining keeps them in a short list.



Q4: Time Complexity Comparison

- **Linear search**: O(n), up to 50,000 checks.
- **Binary search**: O(log n), about 16 checks, but data must be sorted.
- **Hash table**: O(1) on average, about 1 check.
- Conclusion: the hash table is the most suitable for fast ISBN search.

